Data Mining

Lecture # 9

Mining Frequent Patterns
Associations and Correlations

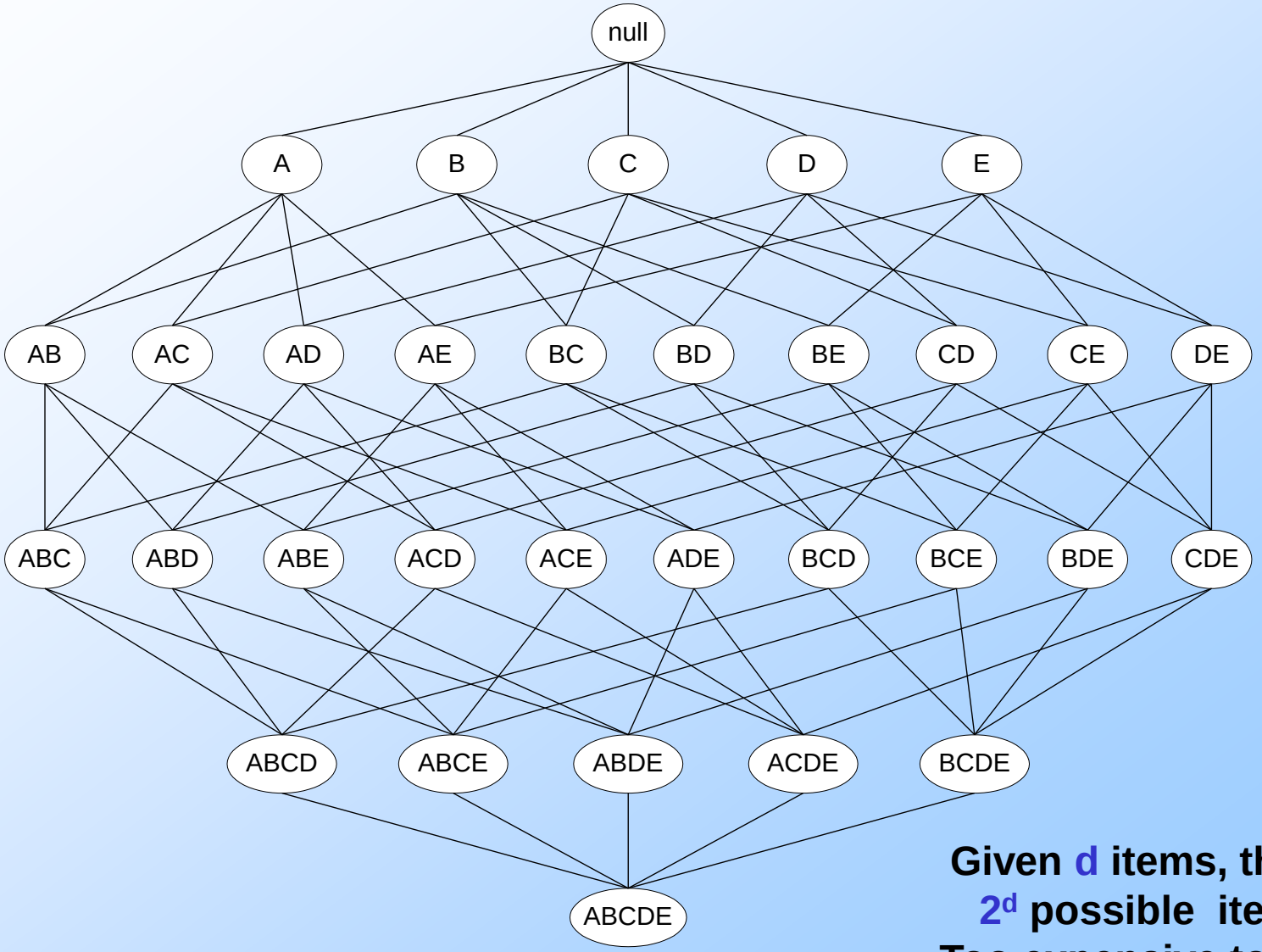
RECAP

Mining Frequent Itemsets

TID	Items
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Coke
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Coke

- **Itemset**
  - A collection of one or more items
    - Example: {Milk, Bread, Diaper}
  - k-itemset
    - An itemset that contains k items
- **Support (σ)**
  - **Count**: Frequency of occurrence of an itemset
  - E.g. σ({Milk, Bread,Diaper}) = 2
  - **Fraction**: Fraction of transactions that contain an itemset
  - E.g. s({Milk, Bread, Diaper}) = 40%
- **Frequent Itemset**
  - An itemset whose support is greater than or equal to a minsup threshold, $s(I) \geq$ minsup
- **Problem Definition**
  - Input: A set of transactions T, over a set of items I, minsup value
  - Output: All itemsets with items in I having $s(I) \geq$ minsup

The itemset lattice



Given d items, there are 2^d possible itemsets
Too expensive to test all!

The Apriori Principle

Apriori principle (Main observation):

- **If an itemset is frequent, then all of its subsets must also be frequent**
- **If an itemset is not frequent, then all of its supersets cannot be frequent**

$$\forall X, Y : (X \subseteq Y) \Rightarrow s(X) \geq s(Y)$$

- **The support of an itemset never exceeds the support of its subsets**
- **This is known as the anti-monotone property of support**

Illustration of the Apriori principle

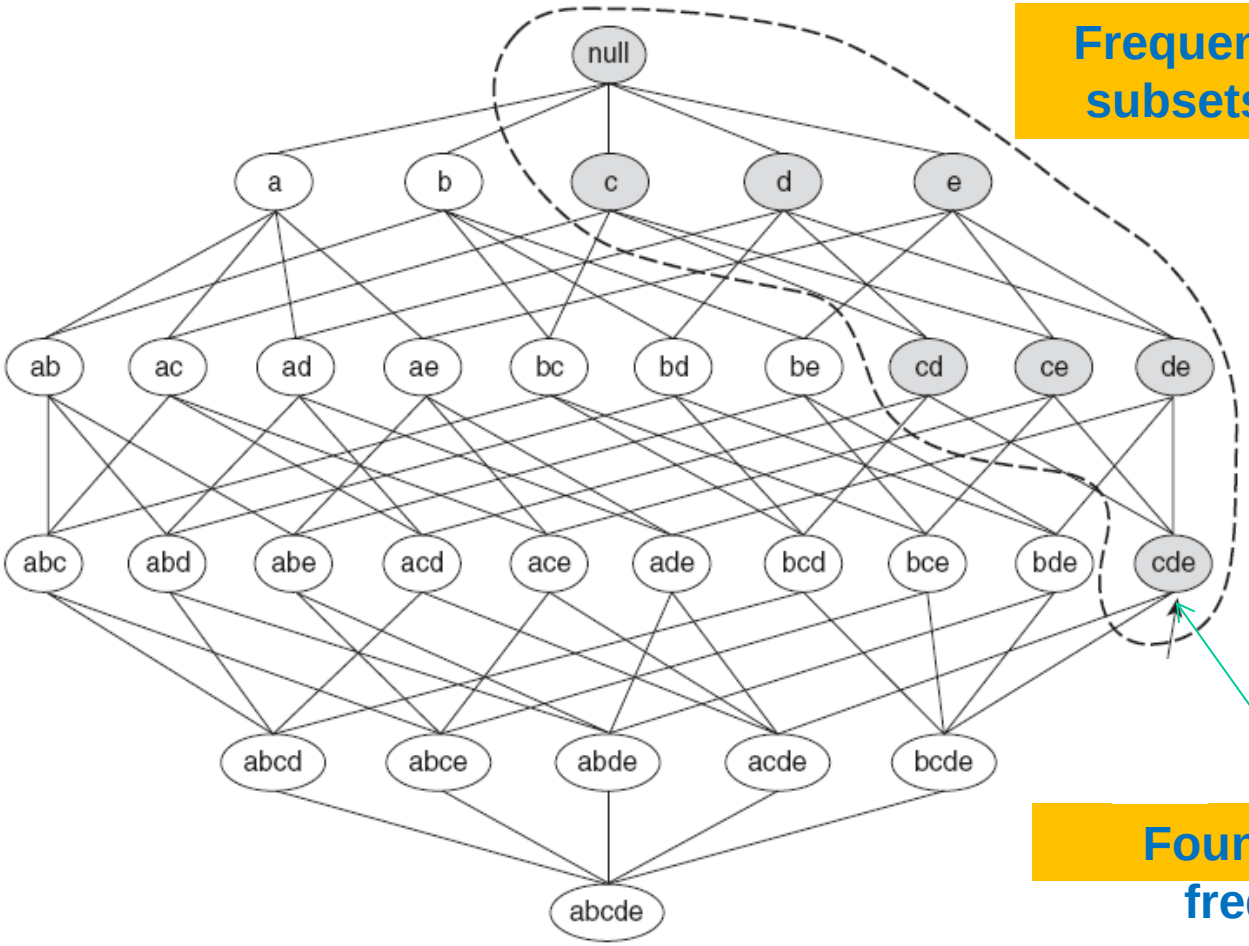
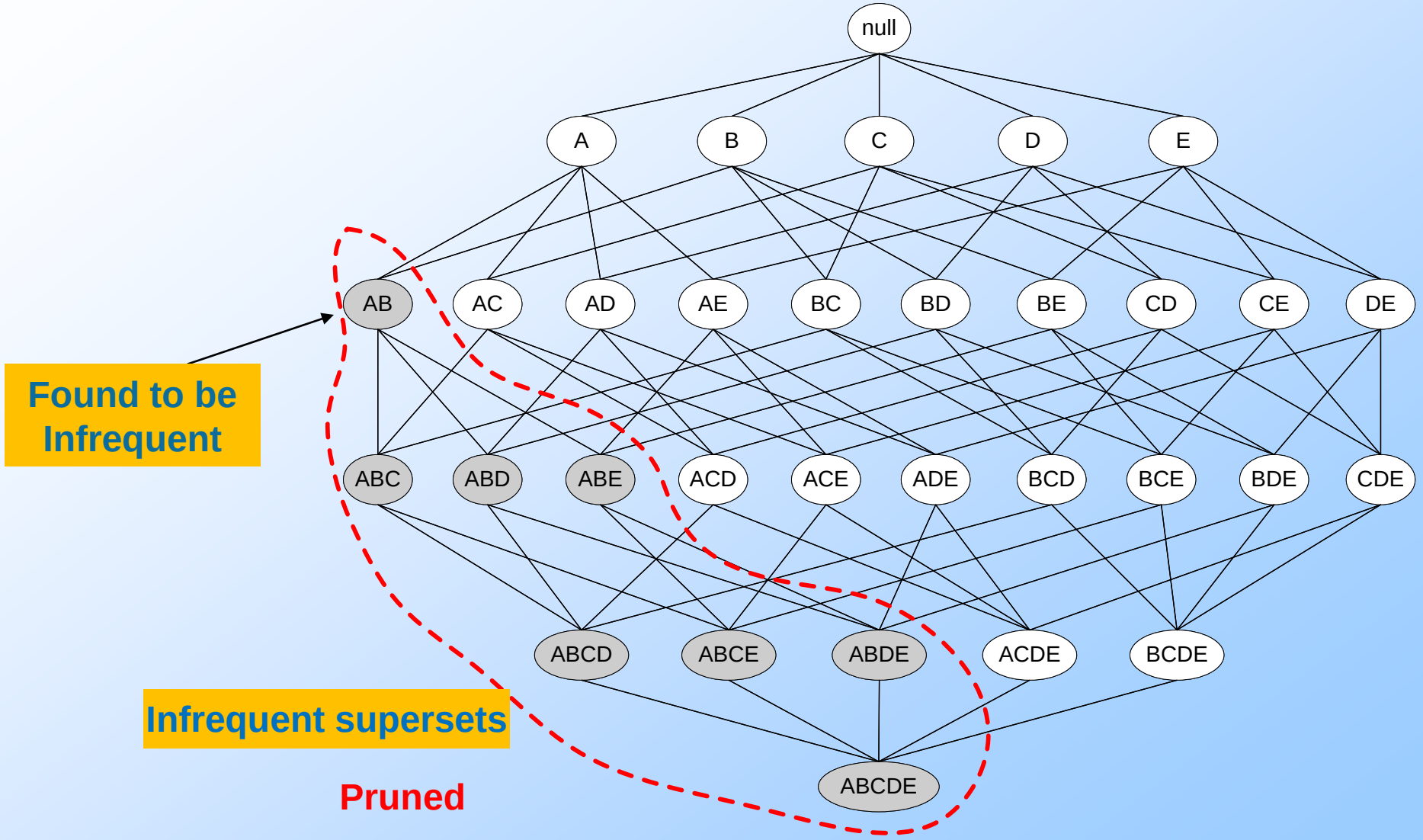


Figure 6.3. An illustration of the *Apriori* principle. If $\{c, d, e\}$ is frequent, then all subsets of this itemset are frequent.

Illustration of the Apriori principle



The Apriori algorithm

Level-wise approach

C_k = candidate itemsets of size k
L_k = frequent itemsets of size k

1. k = 1, C_1 = all items
2. While C_k not empty

Frequent itemset generation

3. Scan the database to find which itemsets in C_k are frequent and put them into L_k

Candidate generation

4. Use L_k to generate a collection of candidate itemsets C_{k+1} of size k+1

5. k = k+1

R. Agrawal, R. Srikant: "Fast Algorithms for Mining Association Rules", *Proc. of the 20th Int'l Conference on Very Large Databases*, 1994.

Candidate Generation

- Basic principle (Apriori):
  - An itemset of size k+1 is candidate to be frequent only if all of its subsets of size k are known to be frequent
- Main idea:
  - Construct a candidate of size k+1 by combining two frequent itemsets of size k
  - Prune the generated k+1-itemsets that do not have all k-subsets to be frequent

Mining Frequent Patterns without Candidate Generation

Slides Modified From Song Wang Mohammed and Zhenyu's Version

Outline

- Frequent Pattern Mining: Problem statement and an example
- Review of Apriori-like Approaches
- FP-Growth:
  - Overview
  - FP-tree:
    - structure, construction and advantages
  - FP-growth:
    - FP-tree →conditional pattern bases → conditional FP-tree →frequent patterns
- Experiments
- Discussion:
  - Improvement of FP-growth
- Conclusion Remarks

Frequent Pattern Mining: An Example

Given a transaction database DB and a minimum support threshold δ, find all frequent patterns (item sets) with support no less than δ.

Input: DB:

TID	*Items bought*
100	{*f*, *a*, *c*, *d*, *g*, *i*, *m*, *p*}
200	{*a*, *b*, *c*, *f*, *l*, *m*, *o*}
300	{*b*, *f*, *h*, *j*, *o*}
400	{*b*, *c*, *k*, *s*, *p*}
500	{*a*, *f*, *c*, *e*, *l*, *p*, *m*, *n*}

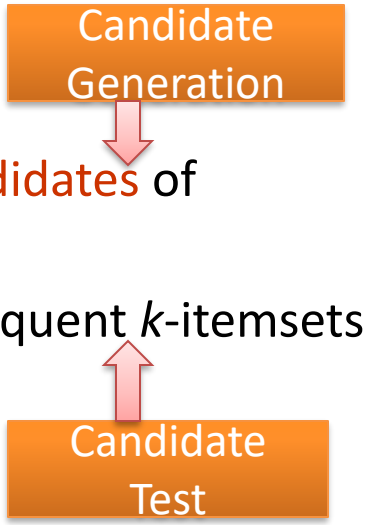
Minimum support: δ =3

Output: all frequent patterns, i.e., *f*, *a*, …, *fa*, *fac*, *fam*, *fm*,*am*…

Problem Statement: How to **efficiently** find all frequent patterns?

Apriori

- Main Steps of Apriori Algorithm:
  - Use frequent (k – 1)-itemsets (L_{k-1}) to generate candidates of frequent k-itemsets C_k
  - Scan database and count each pattern in C_k, get frequent k-itemsets (L_k) .
- E.g. ,



TID	*Items bought*
100	{*f, a, c, d, g, i, m, p*}
200	{*a, b, c, f, l, m, o*}
300	{*b, f, h, j, o*}
400	{*b, c, k, s, p*}
500	{*a, f, c, e, l, p, m, n*}

Apriori	*iteration*
C1	*f,a,c,d,g,i,m,p,l,o,h,j,k,s,b,e,n*
L1	*f, a, c, m, b, p*
C2	*fa, fc, fm, fp, ac, am, …bp*
L2	*fa, fc, fm, …*
…	

Performance Bottlenecks of Apriori

- Bottlenecks of *Apriori*: candidate generation
  - Generate huge candidate sets:
    - 10^4 frequent 1-itemset will generate 10^7 candidate 2-itemsets
    - To discover a frequent pattern of size 100, e.g., $\{a_1, a_2, \ldots, a_{100}\}$, one needs to generate $2^{100} \approx 10^{30}$ candidates.
  - Candidate Test incur multiple scans of database: each candidate

Overview of FP-Growth: Ideas

- Compress a large database into a compact, *Frequent-Pattern tree* (FP-tree) structure
  - highly compacted, but complete for frequent pattern mining
  - avoid costly repeated database scans
- Develop an efficient, FP-tree-based frequent pattern mining method (FP-growth)
  - A divide-and-conquer methodology: decompose mining tasks into smaller ones
  - Avoid candidate generation: sub-database test only.

FP-tree:
Construction and Design

Construct FP-tree

Two Steps:

1. Scan the transaction DB for the first time, find frequent items (single item patterns) and order them into a list L in frequency descending order.

   e.g., L={f:4, c:4, a:3, b:3, m:3, p:3}

   In the format of (item-name, support)

2. For each transaction, order its frequent items according to the order in L; Scan DB the second time, construct FP-tree by putting each frequency ordered transaction onto it.

FP-tree Example: step 1

Step 1: Scan DB for the first time to generate L

TID	*Items bought*
100	{*f*, *a*, *c*, *d*, *g*, *i*, *m*, *p*}
200	{*a*, *b*, *c*, *f*, *l*, *m*, *o*}
300	{*b*, *f*, *h*, *j*, *o*}
400	{*b*, *c*, *k*, *s*, *p*}
500	{*a*, *f*, *c*, *e*, *l*, *p*, *m*, *n*}

L

Item	*frequency*
f	*4*
c	*4*
a	*3*
b	*3*
m	*3*
p	*3*

By-Product of First Scan of Database

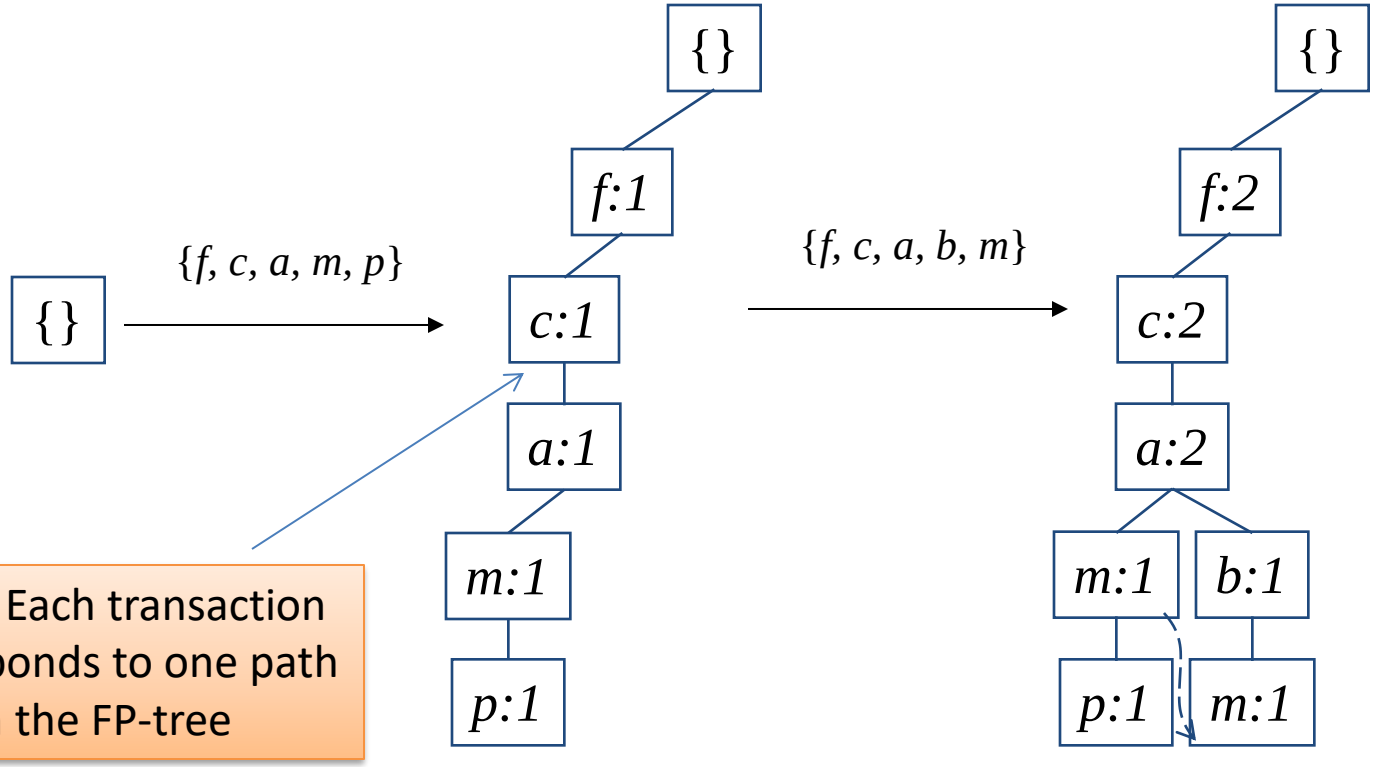
FP-tree Example: step 2

Step 2: scan the DB for the second time, order frequent items in each transaction

TID	*Items bought*	*(ordered) frequent items*
100	{*f*, *a*, *c*, *d*, *g*, *i*, *m*, *p*}	{*f*, *c*, *a*, *m*, *p*}
200	{*a*, *b*, *c*, *f*, *l*, *m*, *o*}	{*f*, *c*, *a*, *b*, *m*}
300	{*b*, *f*, *h*, *j*, *o*}	{*f*, *b*}
400	{*b*, *c*, *k*, *s*, *p*}	{*c*, *b*, *p*}
500	{*a*, *f*, *c*, *e*, *l*, *p*, *m*, *n*}	{*f*, *c*, *a*, *m*, *p*}

FP-tree Example: step 2

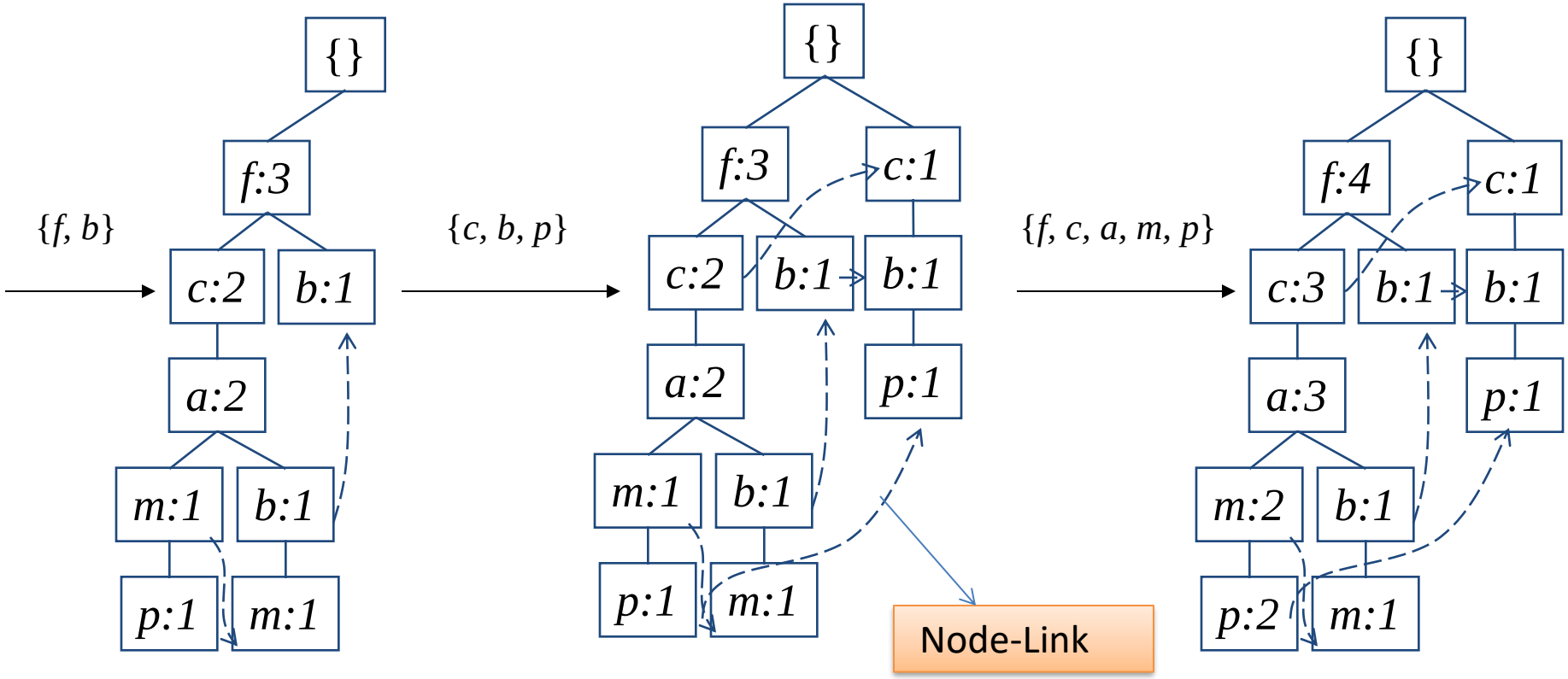
Step 2: construct FP-tree



NOTE: Each transaction corresponds to one path in the FP-tree

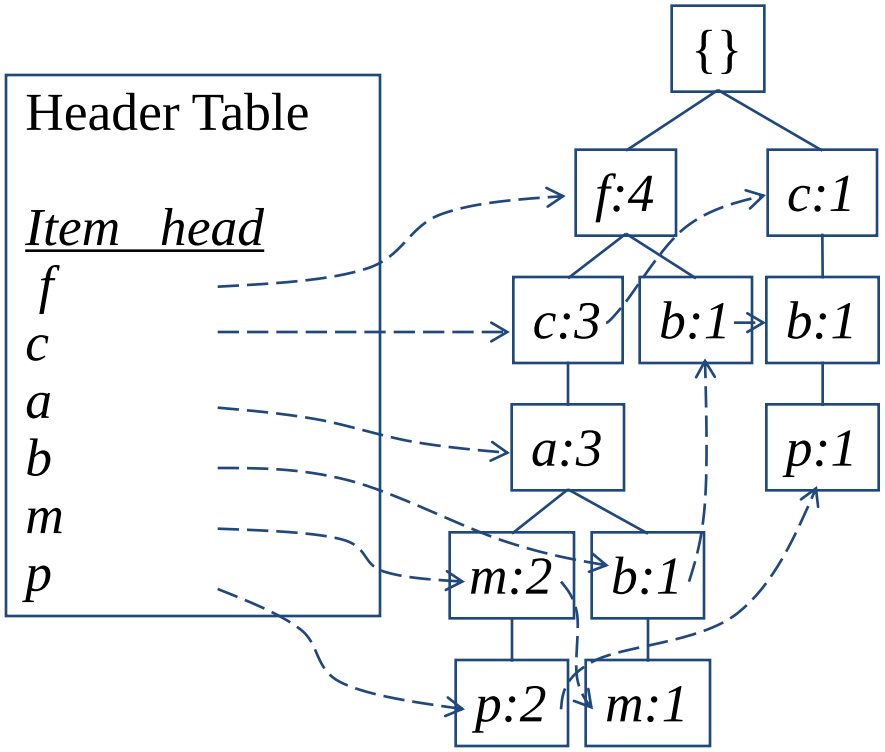
FP-tree Example: step 2

Step 2: construct FP-tree



Construction Example

Final FP-tree



FP-Tree Definition

- FP-tree is **a frequent pattern tree** . Formally, FP-tree is a tree structure defined below:

  1. One root labeled as “null", a set of *item prefix sub-trees* as the children of the root, and a *frequent-item header table*.

  2. Each node in *the item prefix sub-trees* has three fields:
     - item-name : register which item this node represents,
     - count, the number of transactions represented by the portion of the path reaching this node,
     - node-link that links to the next node in the FP-tree carrying the same item-name, or null if there is none.

  3. Each entry in the *frequent-item header table* has two fields,
     - item-name, and
     - head of node-link that points to the first node in the FP-tree carrying the item-name.

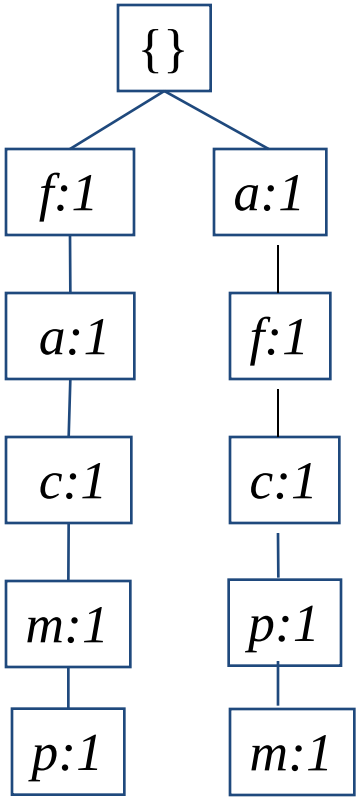
Advantages of the FP-tree Structure

- The most significant advantage of the FP-tree
  - Scan the DB only twice and twice only.
- Completeness:
  - the FP-tree contains all the information related to mining frequent patterns (given the min-support threshold). Why?
- Compactness:
  - The size of the tree is bounded by the occurrences of frequent items
  - The height of the tree is bounded by the maximum number of items in a transaction

Questions?

- Why descending order?
- Example 1:

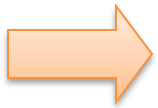
TID	*(unordered) frequent items*
100	$\{f, a, c, m, p\}$
500	$\{a, f, c, p, m\}$

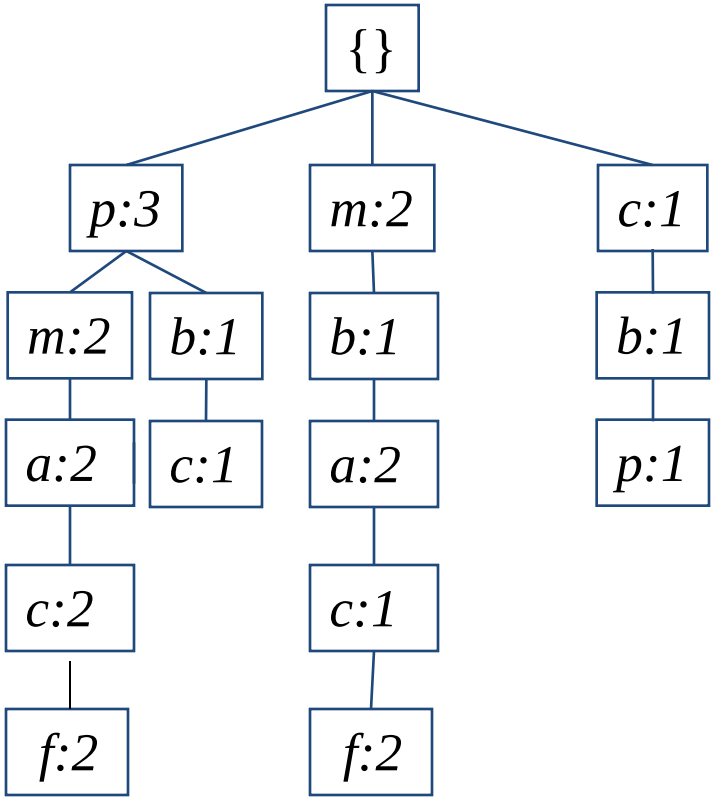


Questions?

- Example 2:

TID	*(ascended) frequent items*
100	{*p*, *m*, *a*, *c*, *f*}
200	{*m*, *b*, *a*, *c*, *f*}
300	{*b*, *f*}
400	{*p*, *b*, *c*}
500	{*p*, *m*, *a*, *c*, *f*}





This tree is larger than FP-tree, because in FP-tree, more frequent items have a higher position, which makes branches less

FP-growth:
Mining Frequent Patterns Using FP-tree

Mining Frequent Patterns Using FP-tree

- General idea (divide-and-conquer)

  Recursively grow frequent patterns using the FP-tree: looking for shorter ones recursively and then concatenating the suffix:

  – For each frequent item, construct its conditional pattern base, and then its conditional FP-tree;

  – Repeat the process on each newly created conditional FP-tree until the resulting FP-tree is empty, or it contains only one path (single path will generate all the combinations of its sub-paths, each of which is a frequent pattern)

3 Major Steps

Starting the processing from the end of list L:

Step 1:

Construct **conditional pattern base** for each item in the header table

Step 2

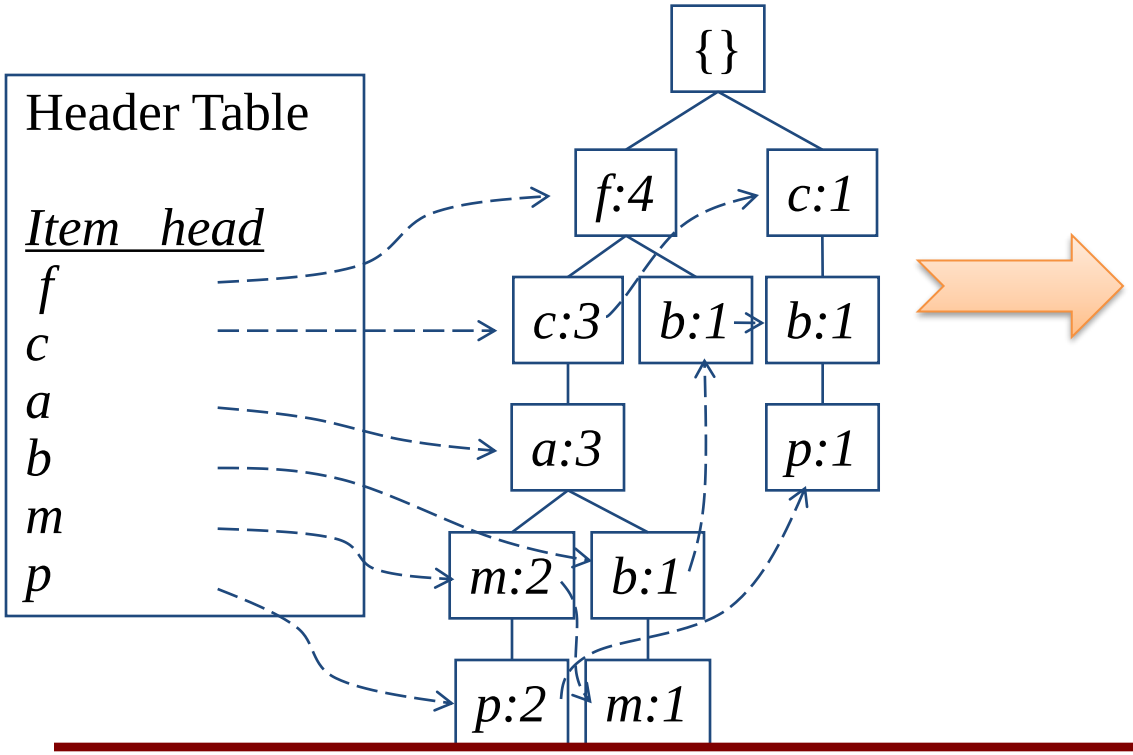
Construct **conditional FP-tree** from each conditional pattern base

Step 3

Recursively mine conditional FP-trees and grow frequent patterns obtained so far. If the conditional FP-tree contains a single path, simply enumerate all the patterns

Step 1: Construct Conditional Pattern Base

- Starting at the bottom of frequent-item header table in the FP-tree
- Traverse the FP-tree by following the link of each frequent item
- Accumulate all of **transformed prefix paths** of that item to form a **conditional pattern base**



***Conditional* pattern bases**

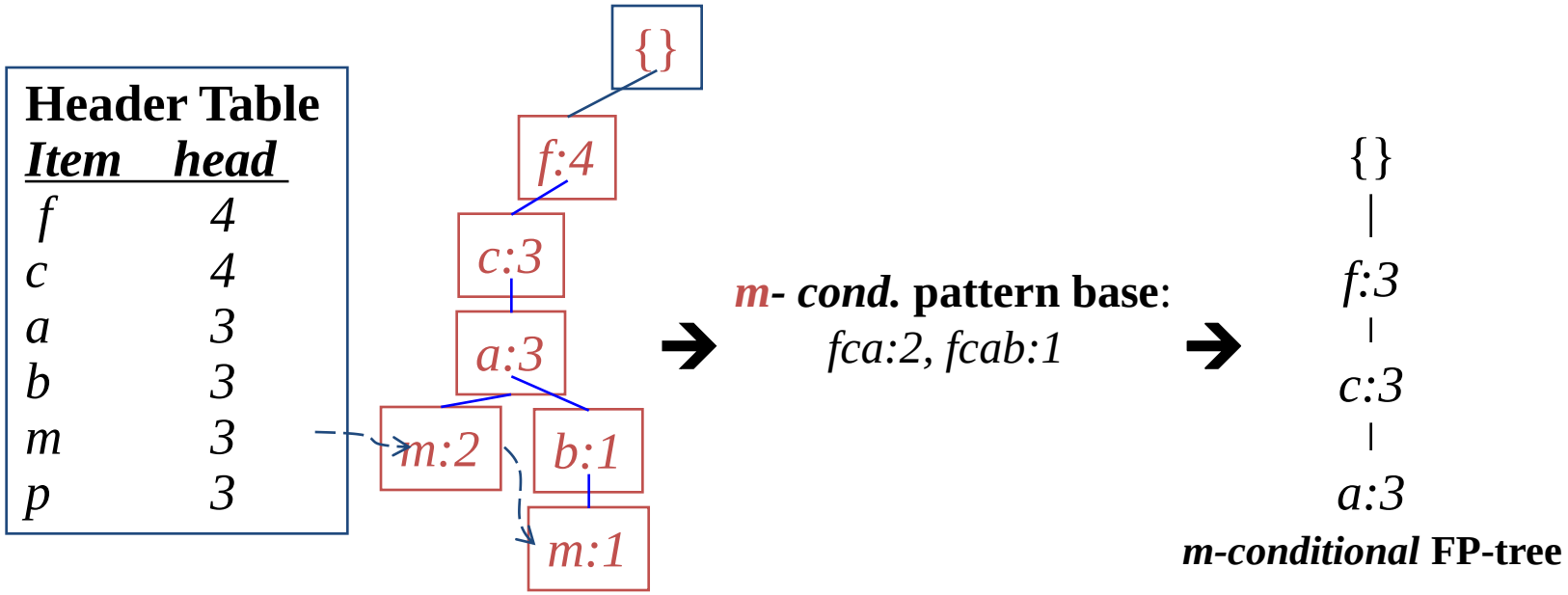
item	*cond. pattern base*
p	***fcam:2, cb:1***
m	***fca:2, fcab:1***
b	***fca:1, f:1, c:1***
a	***fc:3***
c	***f:3***
f	***{}***

Properties of FP-Tree

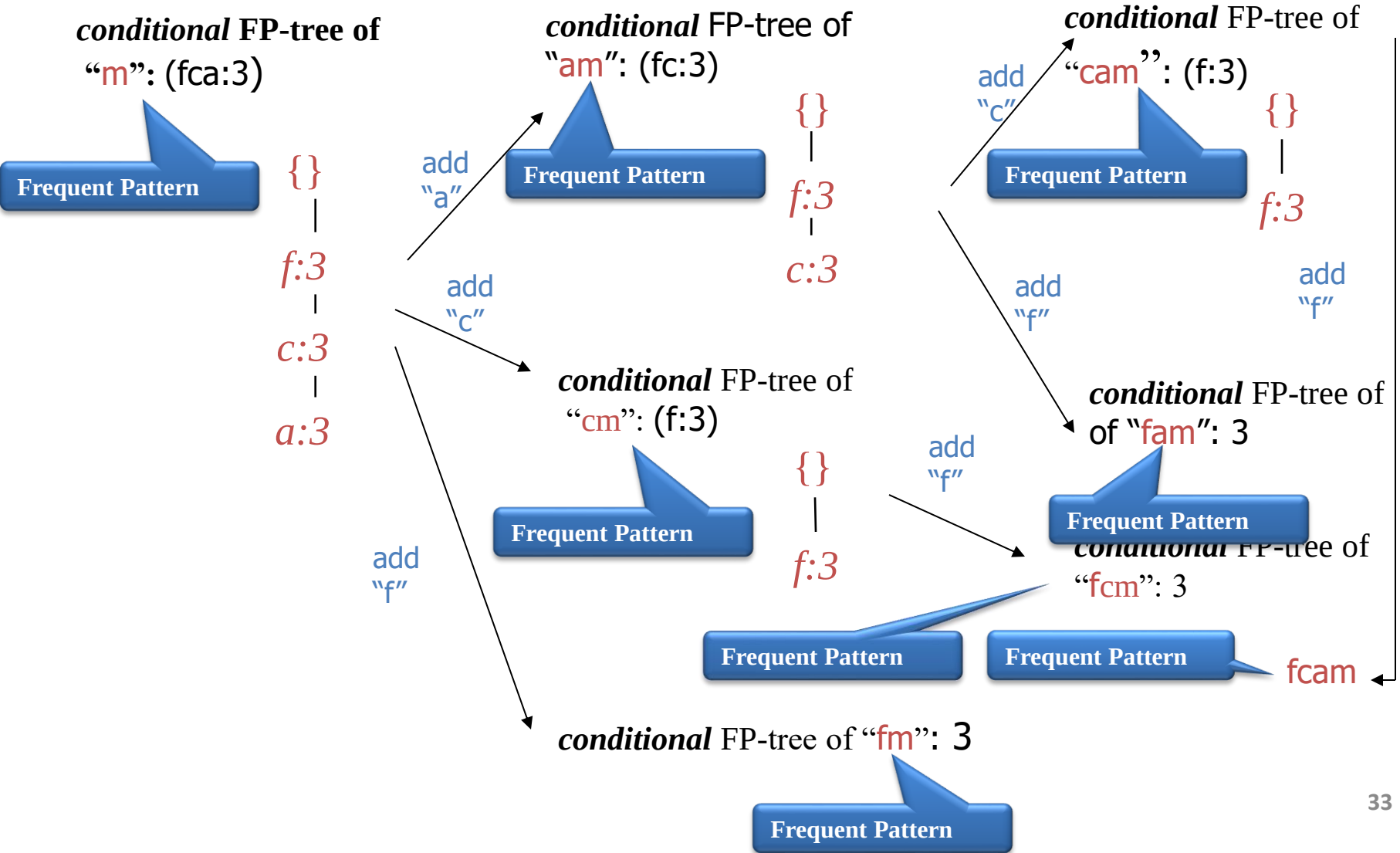
- Node-link property
  - For any frequent item a_i, all the possible frequent patterns that contain a_i can be obtained by following a_i's node-links, starting from a_i's head in the FP-tree header.
- Prefix path property
  - To calculate the frequent patterns for a node a_i in a path P, only the prefix sub-path of a_i in P need to be accumulated, and its frequency count should carry the same count as node a_i.

Step 2: Construct Conditional FP-tree

- For each pattern base
  - Accumulate the count for each item in the base
  - Construct the conditional FP-tree for the frequent items of the pattern base



Step 3: Recursively mine the conditional FP-tree



Principles of FP-Growth

- Pattern growth property
  - Let α be a frequent itemset in DB, B be α's conditional pattern base, and β be an itemset in B. Then $\alpha \cup \beta$ is a frequent itemset in DB iff β is frequent in B.
- Is "*fcabm* " a frequent pattern?
  - "fcab" is a branch of m's conditional pattern base
  - "b" is **NOT** frequent in transactions containing "fcab "
  - "bm" is **NOT** a frequent itemset.

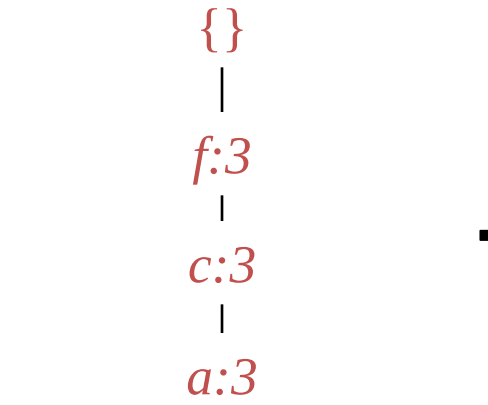
Conditional Pattern Bases and Conditional FP-Tree

Item	Conditional pattern base	Conditional FP-tree
p	{(fcam:2), (cb:1)}	{(c:3)}|p
m	{(fca:2), (fcab:1)}	{(f:3, c:3, a:3)}|m
b	{(fca:1), (f:1), (c:1)}	Empty
a	{(fc:3)}	{(f:3, c:3)}|a
c	{(f:3)}	{(f:3)}|c
f	Empty	Empty

order of L

Single FP-tree Path Generation

- Suppose an FP-tree T has a single path P. The complete set of frequent pattern of T can be generated by enumeration of all the combinations of the sub-paths of P



m-conditional FP-tree

→

All frequent patterns concerning *m*: combination of {f, c, a} and *m*

m,

fm, *cm*, *am*,

fcm, *fam*, *cam*,

fcam

Summary of FP-Growth Algorithm

- Mining frequent patterns can be viewed as first mining 1-itemset and progressively growing each 1-itemset by mining on its conditional pattern base recursively

- Transform a frequent k-itemset mining problem into a sequence of k frequent 1-itemset mining problems via a set of conditional pattern bases

Efficiency Analysis

Facts: usually

1. FP-tree is much smaller than the size of the DB
2. Pattern base is smaller than original FP-tree
3. Conditional FP-tree is smaller than pattern base

➔ mining process works on a set of usually much smaller pattern bases and conditional FP-trees

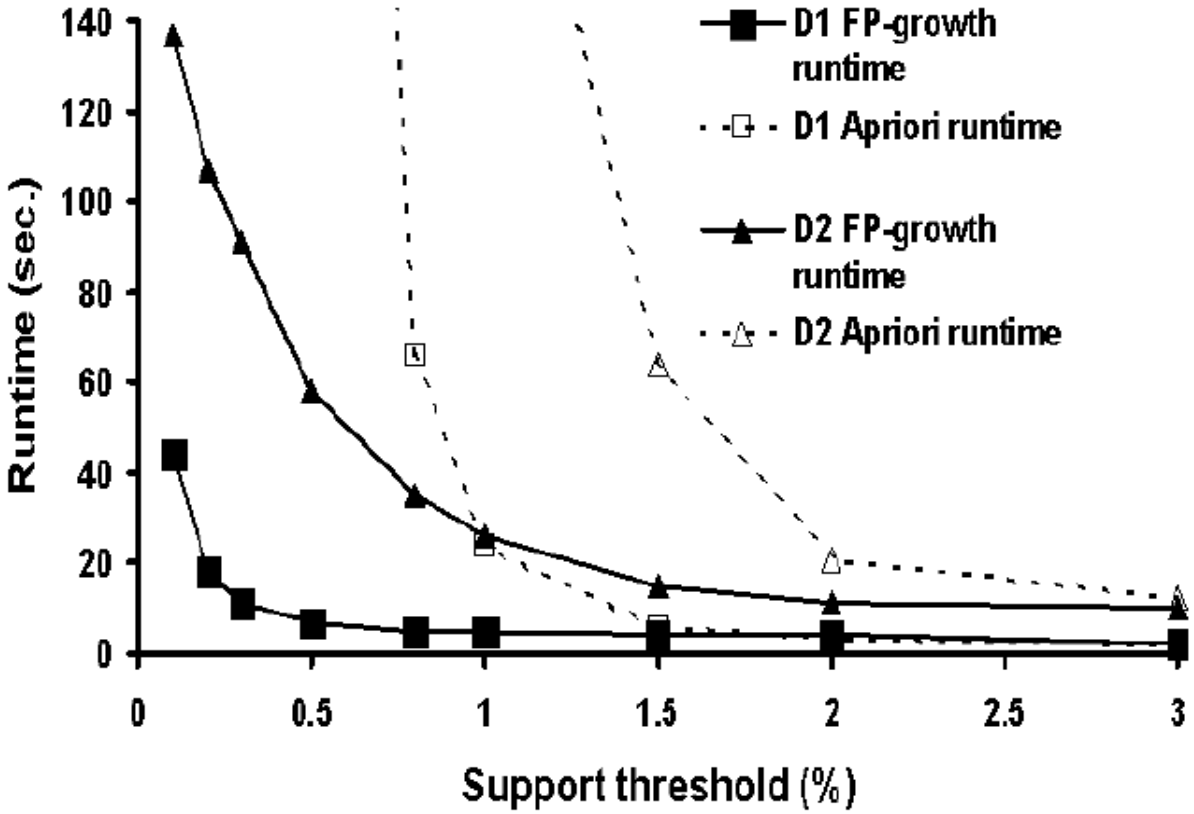
➔ Divide-and-conquer and dramatic scale of shrinking

Experiments:
Performance Evaluation

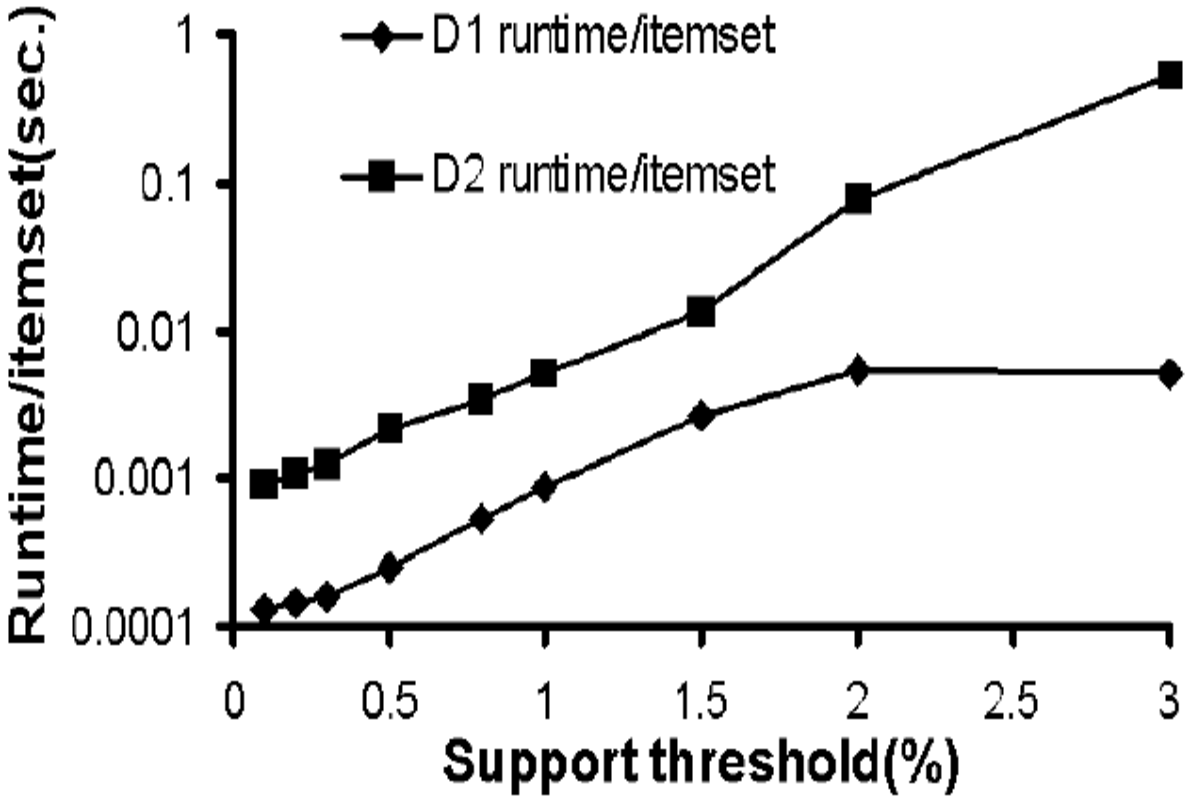
Experiment Setup

- Compare the runtime of FP-growth with classical Apriori and recent TreeProjection
  - Runtime vs. min_sup
  - Runtime per itemset vs. min_sup
  - Runtime vs. size of the DB (# of transactions)
- Synthetic data sets : frequent itemsets grows exponentially as minisup goes down
  - D1: T25.I10.D10K
    - 1K items
    - avg(transaction size)=25
    - avg(max/potential frequent item size)=10
    - 10K transactions
  - D2: T25.I20.D100K
    - 10k items

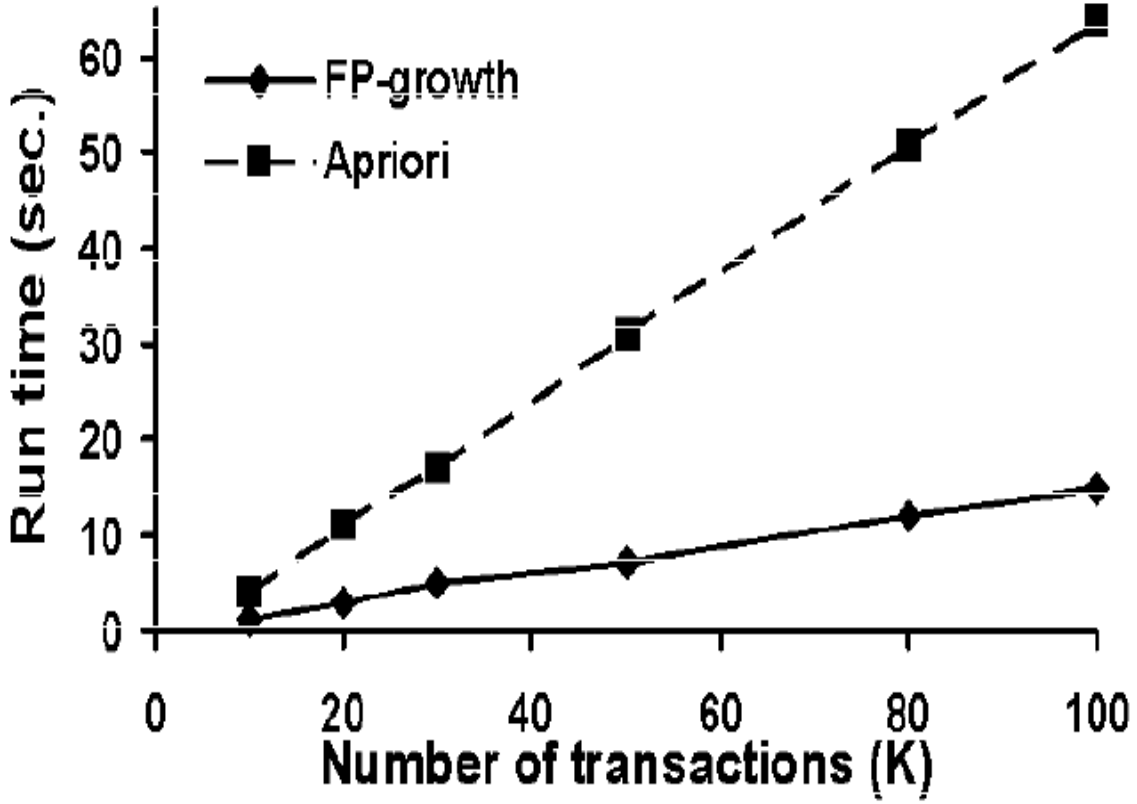
Scalability: runtime vs. min_sup (w/ Apriori)



Runtime/itemset vs. min_sup

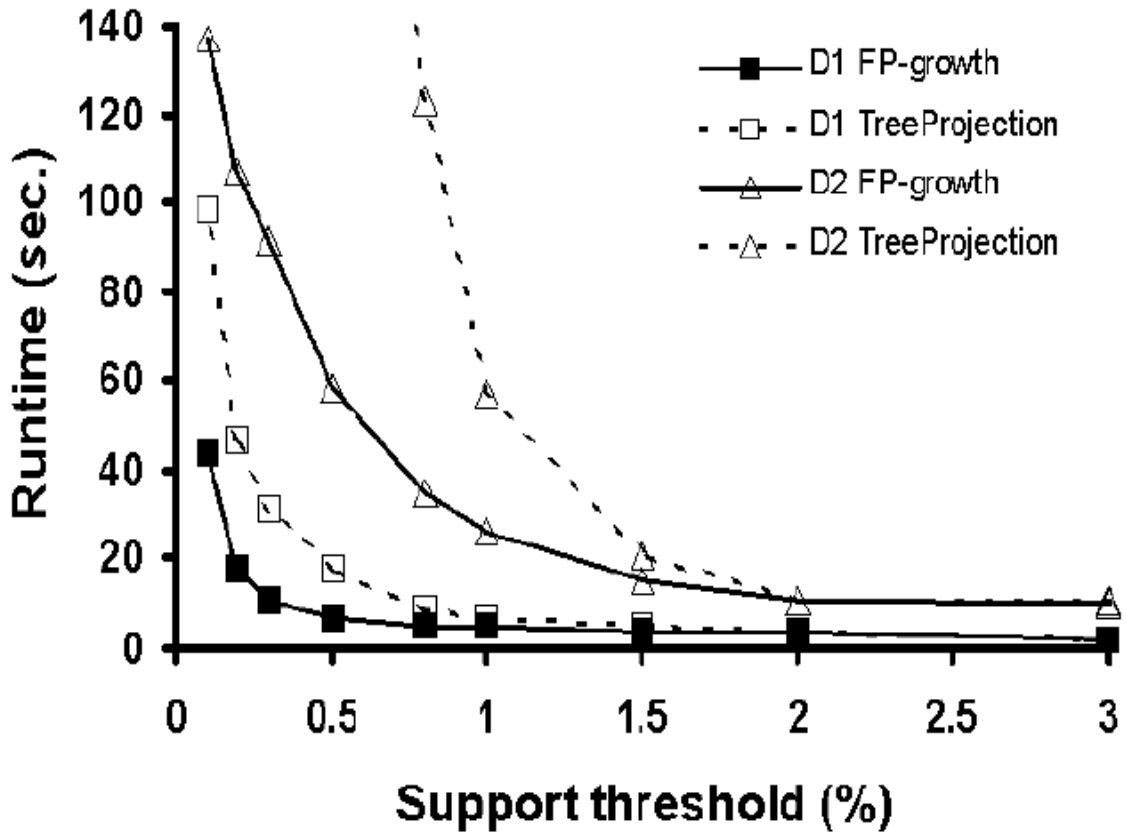


Scalability: runtime vs. # of Trans. (w/ Apriori)

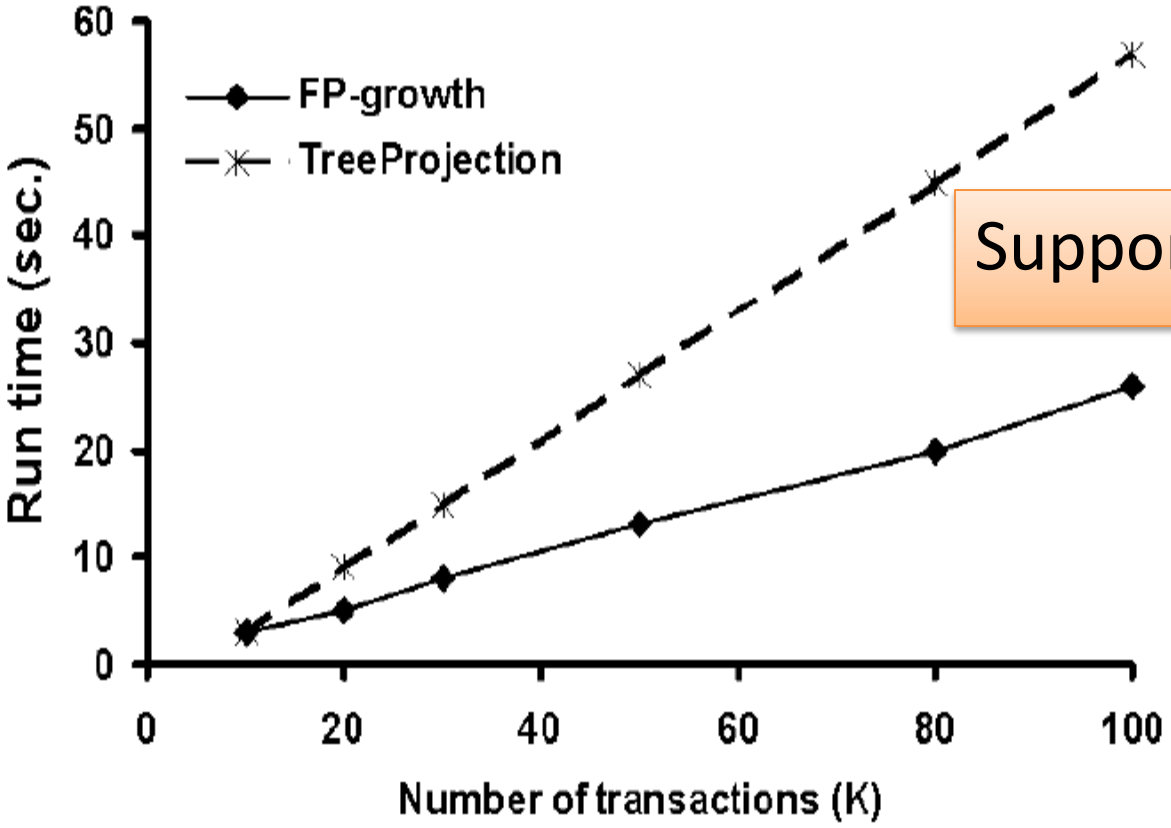


* Using D2 and min_support=1.5%

Scalability: runtime vs. min_support (w/ TreeProjection)



Scalability: runtime vs. # of Trans. (w/ TreeProjection)

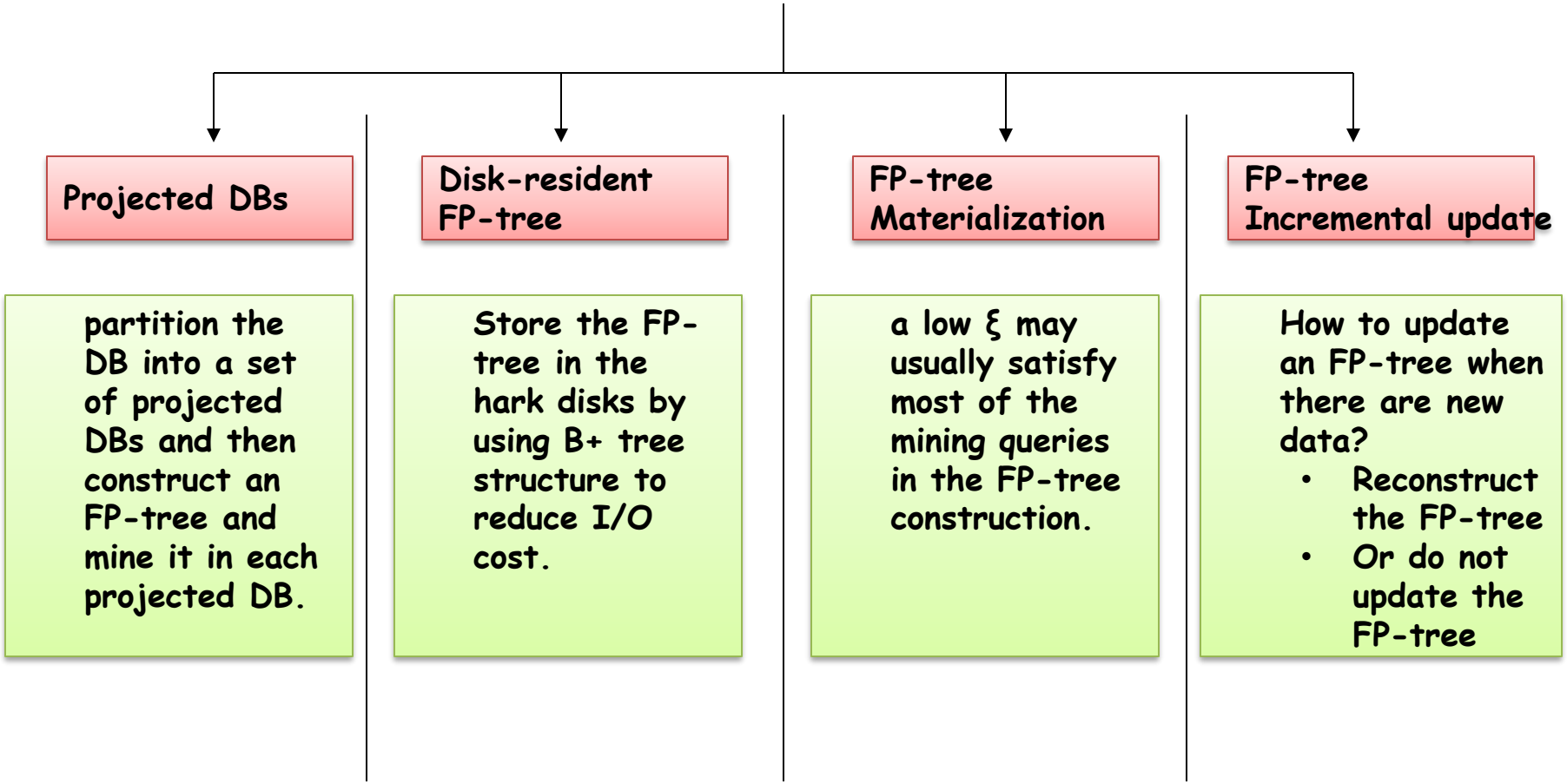


Support = 1%

Discussions:

Improve the performance and scalability of FP-growth

Performance Improvement



Conclusion Remarks

- FP-tree: a novel data structure storing compressed, crucial information about frequent patterns, compact yet complete for frequent pattern mining.

- FP-growth: an efficient mining method of frequent patterns in large Database: using a highly compact FP-tree, divide-and-conquer method in nature.

Some Notes

- In association analysis, there are two main steps, find complete frequent patterns is the first step, though more important step;
- Both Apriori and FP-Growth are aiming to find out complete set of patterns;
- FP-Growth is more efficient and scalable than Apriori in respect to prolific and long patterns.

Related info.

- FP_growth method is (year 2000) available in DBMiner.
- Original paper appeared in SIGMOD 2000. The extended version was just published: "**Mining Frequent Patterns without Candidate Generation: A Frequent-Pattern Tree Approach**" *Data Mining and Knowledge Discovery, 8, 53–87,* 2004. Kluwer Academic Publishers.
- Textbook: "**Data Ming: Concepts and Techniques**" Chapter 6.2.4 (Page 239~243)

Exams Questions

- ***Q1:*** *What are the main drawback s of Apriori –like approaches and explain why ?*
- **A:**
- The main disadvantages of Apriori-like approaches are:
  1. It is costly to generate those candidate sets;
  2. It incurs multiple scan of the database.

The reason is that: Apriori is based on the following heuristic/down-closure property:

if any length *k* patterns is not frequent in the database, any length *(k+1)* super-pattern can never be frequent.

The two steps in Apriori are candidate generation and test. If the 1-itemsets is huge in the database, then the generation for successive item-sets would be quite costly and thus the test.

Exams Questions

- ***Q2:*** *What is FP-Tree?*
- **Previous answer:** A *FP-Tree* is a tree data structure that represents the database in a compact way. It is constructed by mapping each frequency ordered transaction onto a path in the *FP-Tree*.
- **My Answer**: A FP-Tree is an extended prefix tree structure that represents the transaction database in a compact and complete way. Only frequent length-1 items will have nodes in the tree, and the tree nodes are arranged in such a way that more frequently occurring nodes will have better chances of sharing nodes than less frequently occurring ones. Each transaction in the database is mapped to one path in the FP-Tree.

Exams Questions

- *Q3: What is the most significant advantage of FP-Tree? Why FP-Tree is complete in relevance to frequent pattern mining?*
- A: Efficiency, the most significant advantage of the *FP-tree* is that it requires two scans to the underlying database (and only two scans) to construct the FP-tree. This efficiency is further apparent in database with prolific and long patterns or for mining frequent patterns with low support threshold.
- As each transaction in the database is mapped to one path in the FP-Tree, therefore, the frequent item-set information in each transaction is completely stored in the FP-Tree. Besides, one path in the FP-Tree may represent frequent item-sets in multiple transactions without ambiguity since the path representing every transaction must start from the root of each item prefix sub-tree.

Transactional Data for an *AllElectronics* Branch

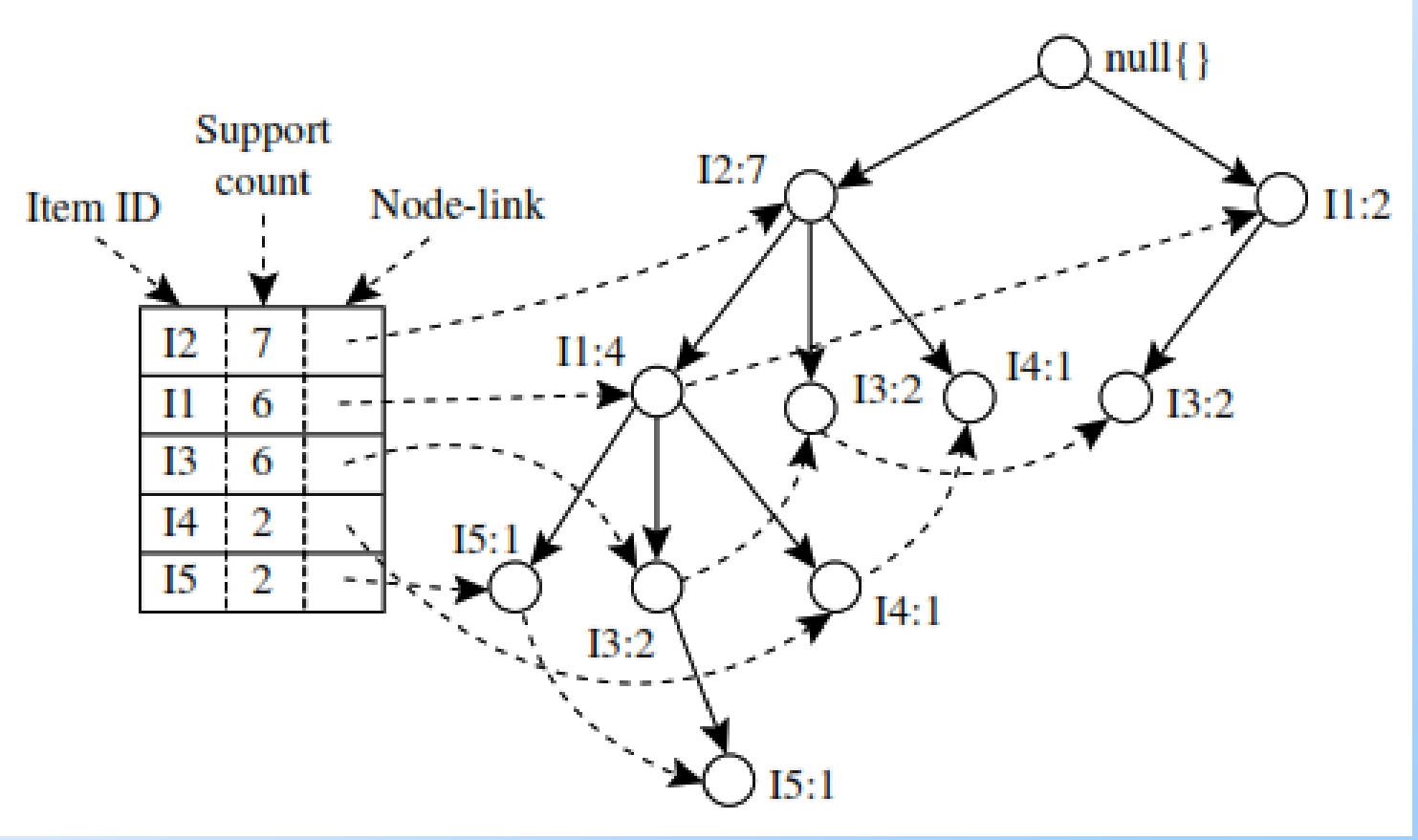
TID	*List of item_IDs*
T100	I1, I2, I5
T200	I2, I4
T300	I2, I3
T400	I1, I2, I4
T500	I1, I3
T600	I2, I3
T700	I1, I3
T800	I1, I2, I3, I5
T900	I1, I2, I3

L: I2, I1, I3, I4, I5

Transaction ID	Item set	Ordered frequent item	
T100	I1, I2, I5	I2, I1, I5	
T200	I2, I4	I2, I4	
T300	I2, I3	I2, I3	
T400	I1, I2, I4	I2, I1, I4	
T500	I1, I3	I1, I3	
T600	I2, I3	I2, I3	
T700	I1, I3	I1, I3	
T800	I1, I2, I3, I5	I2, I1, I3, I5	
T900	I1, I2, I3	I2, I1, I3	

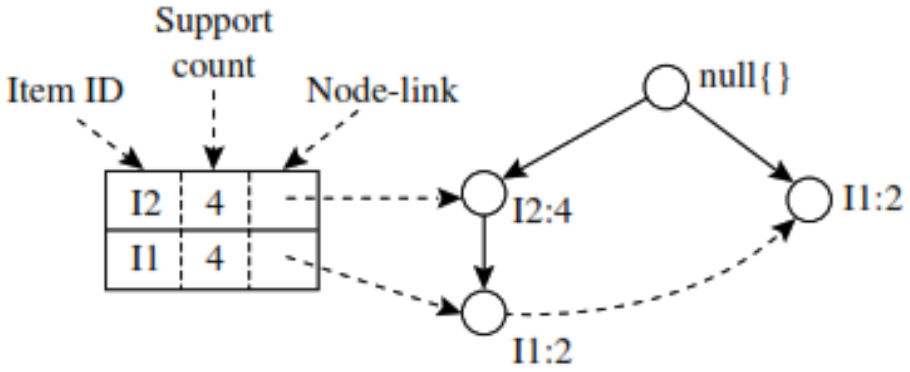
L: I2, I1, I3, I4, I5

FP-Tree


Item ID
Support count
Node-link
I2 7
I1 6
I3 6
I4 2
I5 2
null{ }
I2:7
I1:2
I1:4
I3:2
I4:1
I3:2
I5:1
I3:2
I4:1
I5:1

Frequent pattern mining

Item	*Conditional Pattern Base*	*Conditional FP-tree*	*Frequent Patterns Generated*
I5	{{I2, I1: 1}, {I2, I1, I3: 1}}	⟨I2: 2, I1: 2⟩	{I2, I5: 2}, {I1, I5: 2}, {I2, I1, I5: 2}
I4	{{I2, I1: 1}, {I2: 1}}	⟨I2: 2⟩	{I2, I4: 2}
I3	{{I2, I1: 2}, {I2: 2}, {I1: 2}}	⟨I2: 4, I1: 2⟩, ⟨I1: 2⟩	{I2, I3: 4}, {I1, I3: 4}, {I2, I1, I3: 2}
I1	{{I2: 4}}	⟨I2: 4⟩	{I2, I1: 4}



Mining Association Rules

- **Association Rule**
  - An implication expression of the form $X \rightarrow Y$, where X and Y are itemsets
    - {Milk, Diaper} → {Beer}
- **Rule Evaluation Metrics**
  - Support (s)
    - Fraction of transactions that contain both X and Y **= the probability P(X,Y) that X and Y occur together**
  - Confidence (c)
    - How often Y appears in transactions that contain X = **the conditional probability P(Y|X) that Y occurs given that X has occurred.**

TID	*Items*
1	**Bread, Milk**
2	**Bread, Diaper, Beer, Eggs**
3	**Milk, Diaper, Beer, Coke**
4	**Bread, Milk, Diaper, Beer**
5	**Bread, Milk, Diaper, Coke**

Example:

$$\{\text{Milk}, \text{Diaper}\} \Rightarrow \text{Beer}$$

$$s = \frac{\sigma(\text{Milk}, \text{Diaper}, \text{Beer})}{|\text{T}|} = \frac{2}{5} = 0.4$$

$$c = \frac{\sigma(\text{Milk}, \text{Diaper}, \text{Beer})}{\sigma(\text{Milk}, \text{Diaper})} = \frac{2}{3} = 0.67$$

- **Problem Definition**
  - **Input A set of transactions T, over a set of items I, minsup, minconf values**
  - **Output: All rules with items in I having s ≥ minsup and c≥ minconf**

Mining Association Rules

- Two-step approach:
  1. Frequent Itemset Generation
     - Generate all itemsets whose support $\geq$ minsup
  2. Rule Generation
     - Generate high confidence rules from each frequent itemset, where each rule is a partitioning of a frequent itemset into Left-Hand-Side (LHS) and Right-Hand-Side (RHS)

Frequent itemset: {A,B,C,D}

Rule: AB$\rightarrow$CD

Association Rule anti-monotonicity

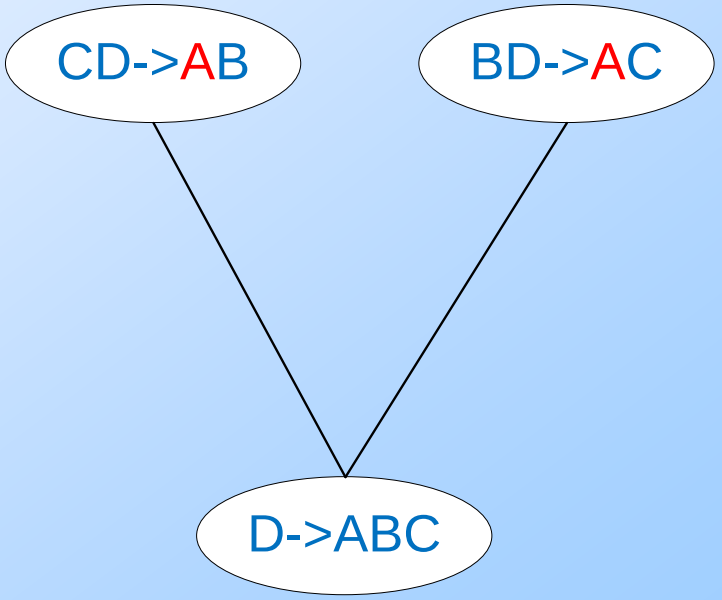
- Confidence is anti-monotone w.r.t. number of items on the RHS of the rule (or monotone with respect to the LHS of the rule)

- e.g., L = {A,B,C,D}:

$$c(ABC \rightarrow D) \geq c(AB \rightarrow CD) \geq c(A \rightarrow BCD)$$

Rule Generation for APriori Algorithm

- Candidate rule is generated by merging two rules that share the same prefix in the RHS



- join(CD→AB,BD→AC) would produce the candidate rule D → ABC
- Prune rule D → ABC if its subset AD→BC does not have high confidence
- Essentially we are doing APriori on the RHS

Closed Itemset

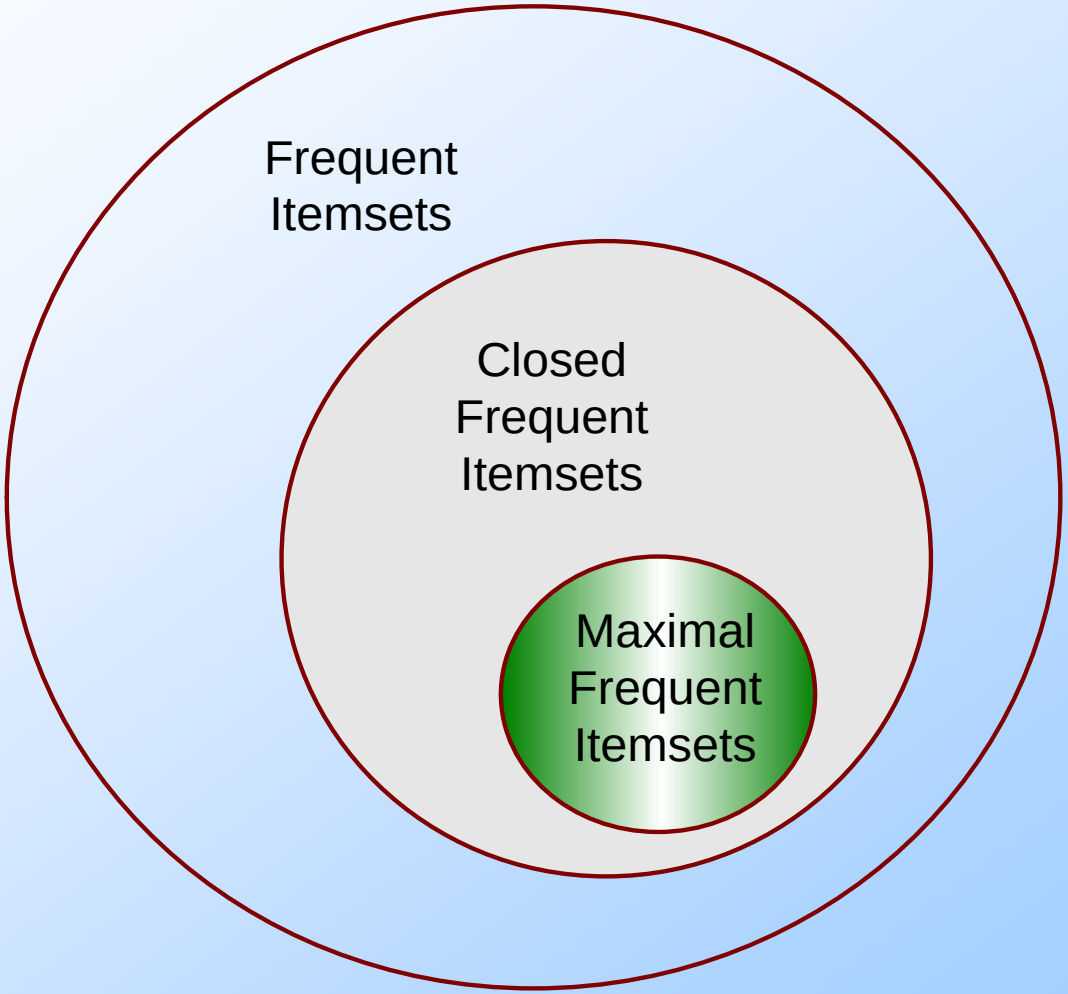
- An itemset is closed if none of its immediate supersets has the same support as the itemset

TID	Items
1	{A,B}
2	{B,C,D}
3	{A,B,C,D}
4	{A,B,D}
5	{A,B,C,D}

Itemset	Support
{A}	4
{B}	5
{C}	3
{D}	4
{A,B}	4
{A,C}	2
{A,D}	3
{B,C}	3
{B,D}	4
{C,D}	3

Itemset	Support
{A,B,C}	2
{A,B,D}	3
{A,C,D}	2
{B,C,D}	3
{A,B,C,D}	2

Maximal vs Closed Itemsets



Pattern Evaluation

- Association rule algorithms tend to produce too many rules but many of them are uninteresting or redundant
  - Redundant if {A,B,C} → {D} and {A,B} → {D} have same support & confidence
    - Summarization techniques
  - Uninteresting, if the pattern that is revealed does not offer useful information.
    - Interestingness measures: a hard problem to define
- Interestingness measures can be used to prune/rank the derived patterns
  - Subjective measures: require human analyst
  - Objective measures: rely on the data.
- In the original formulation of association rules, support & confidence are the only measures used

Computing Interestingness Measure

- Given a rule $X \rightarrow Y$, information needed to compute rule interestingness can be obtained from a contingency table

Contingency table for $X \rightarrow Y$

	Y	$\bar{Y}$	
X	f_{11}	f_{10}	f_{1+}
$\bar{X}$	f_{01}	f_{00}	f_{o+}
	f_{+1}	f_{+0}	N

f_{11}: support of X and Y
f_{10}: support of X and $\bar{Y}$
f_{01}: support of $\bar{X}$ and Y
f_{00}: support of $\bar{X}$ and $\bar{Y}$

X: itemset X appears in tuple
Y: itemset Y appears in tuple
$\bar{X}$: itemset X does not appear in tuple
$\bar{Y}$: itemset Y does not appear in tuple

Used to define various measures

- support, confidence, lift, Gini, J-measure, etc.

Drawback of Confidence

	Coffee	$\overline{\text{Coffee}}$	
Tea	15	5	20
$\overline{\text{Tea}}$	75	5	80
	90	10	100

Number of people that drink tea

Number of people that drink coffee and tea

Number of people that drink coffee but not tea

Number of people that drink coffee

Association Rule: Tea $\rightarrow$ Coffee

Confidence= P(Coffee+Tea|Tea) = $\frac{15}{20} = 0.75$

but P(Coffee) = $\frac{90}{100} = 0.9$

- Although confidence is high, rule is misleading
  - P(Coffee|Tea) = 0.9375

Statistical Independence

- Population of 1000 students
  - 600 students know how to swim (S)
  - 700 students know how to bike (B)
  - 420 students know how to swim and bike (S,B)

  - $P(S \wedge B) = 420/1000 = 0.42$
  - $P(S) \times P(B) = 0.6 \times 0.7 = 0.42$

  - $P(S \wedge B) = P(S) \times P(B)$ => Statistical independence

Statistical Independence

- Population of 1000 students
  - 600 students know how to swim (S)
  - 700 students know how to bike (B)
  - 500 students know how to swim and bike (S,B)

  - $P(S \wedge B) = 500/1000 = 0.5$
  - $P(S) \times P(B) = 0.6 \times 0.7 = 0.42$

  - $P(S \wedge B) > P(S) \times P(B)$ => Positively correlated

Statistical Independence

- Population of 1000 students
  - 600 students know how to swim (S)
  - 700 students know how to bike (B)
  - 300 students know how to swim and bike (S,B)

  - $P(S \wedge B) = 300/1000 = 0.3$
  - $P(S) \times P(B) = 0.6 \times 0.7 = 0.42$

  - $P(S \wedge B) < P(S) \times P(B)$ => Negatively correlated

Statistical-based Measures

- Measures that take into account statistical dependence
  - Lift/Interest/PMI

$$\text{Lift} = \frac{P(Y|X)}{P(Y)} = \frac{P(X,Y)}{P(X)P(Y)} = \text{Interest}$$

In text mining it is called: Pointwise Mutual Information

  - Piatesky-Shapiro

$$\text{PS} = P(X,Y) - P(X)P(Y)$$

- All these measures measure deviation from independence
  - The higher, the better (why?)

Example: Lift/Interest

	Coffee	$\overline{\text{Coffee}}$	
Tea	15	5	20
$\overline{\text{Tea}}$	75	5	80
	90	10	100

Association Rule: Tea $\rightarrow$ Coffee

Confidence= P(Coffee|Tea) = 0.75

but P(Coffee) = 0.9

$\Rightarrow$ Lift = 0.75/0.9= 0.8333 (< 1, therefore is negatively associated)

= 0.15/(0.9*0.2)

Another Example

	of	the	of, the
Fraction of documents	0.9	0.9	0.8

$$P(\text{of}, \text{the}) \approx P(\text{of})P(\text{the})$$

If I was creating a document by picking words randomly, (of, the) have more or less the same probability of appearing together by chance

No correlation

	hong	kong	hong, kong
Fraction of documents	0.2	0.2	0.19

$$P(\text{hong}, \text{kong}) \gg P(\text{hong})P(\text{kong})$$

(hong, kong) have much lower probability to appear together by chance. The two words appear almost always only together

Positive correlation

	obama	karagounis	obama, karagounis
Fraction of documents	0.2	0.2	0.001

$$P(\text{obama}, \text{karagounis}) \ll P(\text{obama})P(\text{karagounis})$$

(obama, karagounis) have much higher probability to appear together by chance. The two words appear almost never together

Negative correlation

Drawbacks of Lift/Interest/Mutual Information

	honk	konk	honk, konk
Fraction of documents	0.0001	0.0001	0.0001

$$MI(honk, konk) = \frac{0.0001}{0.0001 * 0.0001} = 10000$$

	hong	kong	hong, kong
Fraction of documents	0.2	0.2	0.19

$$MI(hong, kong) = \frac{0.19}{0.2 * 0.2} = 4.75$$

Rare co-occurrences are deemed more interesting.
But this is not always what we want